

Design and Verification of AXI2APB Bridge Communication Protocol

Mahmut Efil
Electrical-Electronics Engineering
Abdullah Gul University
Kayseri, Turkey
mahmut.efil@agu.edu.tr

Abstract—The Advanced Microcontroller Bus Architecture (AMBA) 4.0 specifications released by ARM have become the de facto standards for the System on Chip buses. In this study, an AXI2APB Bridge protocol that passes the information from one type of the AMBA protocol to another type without data loss will be designed and verified using SystemVerilog Verification Methodology. The designed RTL is aimed to function as the AXI APB Bridge IP Core, situated in XILINX IP Library. Basically, the proposed bridge interface will provide a link between an AXI4-Lite bus and a low-power APB bus. In this study, VIVADO and QuestaSim Software will be used for design and verification purposes. As a result, the evaluated simulation and hardware results show that the bridge between AXI to APB works properly without any transmission problems such as data loss and supports the read and write transactions between these different buses.

Keywords—SoC, AMBA, AXI4-Lite, APB3, SystemVerilog Verification

I. INTRODUCTION

With the development of the technology, Integrated Circuits (ICs) which are composed of electronic components on a single chip have become very popular in electronic designs due to their low cost, low power consumption, and ease of use. In ICs, Intellectual Property (IP) is a reusable unit of logic that can be used as building blocks within chip designs defined as System on Chip (SoC) designs. In this area, an On-Chip Buses (OCB) are used to interconnect the IP cores and the surrounding interface within an FPGA logic system, and one of the most popular SoC bus peripherals which are widely used in the market is called AMBA, announced by ARM in 2010. The AMBA protocols are used in high-performance embedded microcontrollers and the AMBA 4.0 specifications include the following busses/interfaces[1].

- Advanced eXtensible Interface (AXI)
- Advanced High-performance Bus (AHB)
- Advanced System Bus (ASB)
- Advanced Peripheral Bus (APB)
- Advanced Trace Bus (ATB)

The aim of this paper is to design and validate an AXI2APB Bridge communication protocol using two of the AMBA 4.0 interfaces, AXI4-Lite and APB. First, the protocols will be explained separately, and then the proposed architecture for the bridge communication protocol will be explained further. The structure of the APB bus architecture mentioned above will be the valid for this design.

II. OPERATING PROCESS OF AXI4-LITE PROTOCOL

The Advanced eXtensible Interface (AXI) is a parallel high-performance, synchronous, and high-frequency bus architecture that is designed to communicate between master and slave devices. AXI4-Lite is an AXI4 protocol bus with reduced complexity and structure. It consists of 5 channels which are Read Address, Read Data, Write Address, Write Data, and Write Response based on read and write transactions. The channel signals are shown in table 1 and based on these signals, the process is realized.

Table 1. AXI4-Lite Signal Descriptions [1]

Write address channel	Write data channel	Write response channel	Read address channel	Read data channel
AWVALID	WVALID	BVALID	ARVALID	RVALID
AWREADY	WREADY	BREADY	ARREADY	RREADY
AWADDR	WDATA	BRESP	ARADDR	RDATA
AWPROT	WSTRB		ARPROT	RRESP

A. Write Transactions[1]

In the write transactions, the master activates the Write Address channel (AWADDR) and Write Data channel (WDATA) by asserting AWVALID and WVALID. When the address and data become valid, the slave asserts AWREADY and WREADY signals. This is called the handshake process in the AXI protocol. The BVALID signal is asserted by the slave when the Write response channel is valid and if the BREADY signal is asserted, the system is ready to receive a response. Here the important notice is that the slave must wait for AWVALID, AWREADY, WVALID, and WREADY to be asserted before asserting BVALID. Additionally, sparse data transfer is enabled and controlled using theWSTRB signal. The AXI4-Lite write transaction is shown in figure 1.



Figure 1. Write Transaction of AXI4-Lite Protocol

B. Read Transactions[1]

In the read transactions, the master activates the Read Address channel (ARADDR) and Read Data channel (RDATA) by asserting ARVALID and RREADY which indicates that the master is ready to receive data. Next, the RVALID signal is asserted by the slave indicating a successful read transaction. Here, the important notice is that the slave must wait for ARVALID and ARREADY to be asserted before asserting RVALID. The AXI4-Lite read transaction is shown in figure 2.

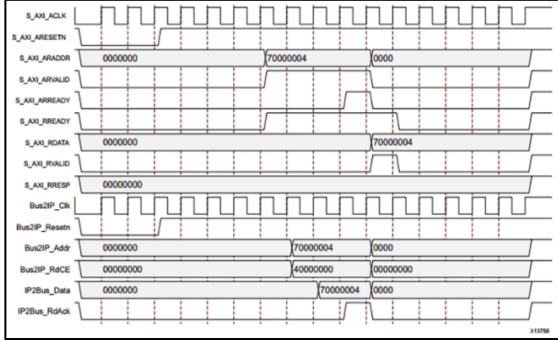


Figure 2. Read Transaction of AXI4-Lite Protocol

III. OPERATING PROCESS OF APB PROTOCOL

The Advanced Peripheral Bus (APB) is a protocol that belongs one of the Advanced Microcontroller Bus Architecture (AMBA) protocol family that is arranged for reduced power consumption and interface complexity. It doesn't include pipelined bus interface and can interface with AHB and AXI busses. In the APB protocol, each write and read transfer takes at least 2 clock cycles. The descriptions for the corresponding signals can be observed at table 2.

Table 2. APB signal descriptions [2]

Signal	Source	Description
PCLK	Clock source	The rising edge of PCL times all transfers on the APB.
PRESETn	System bus equivalent	The APB reset signal is active LOW. This signal is normally connected directly to the system bus reset signal.
PADDR	APB bridge	This is the APB address bus. It can be up to 32 bits wide and is driven by the peripheral bus bridge unit.
PSELx	APB bridge	The APB bridge unit generates this signal to each peripheral bus slave. It indicates that the slave device is selected and that a data transfer is required. There is a PSELx signal for each slave.
PENABLE	APB bridge	This signal indicates the second and subsequent cycles of an APB transfer.
PWRITE	APB bridge	This signal indicates an APB write access when HIGH and an APB read access when LOW.
PWDATA	APB bridge	This bus is driven by the peripheral bus bridge unit during write cycles when PWRITE is HIGH. This bus can be up to 32 bits wide.
PREADY	Slave interface	The slave uses this signal to extend an APB transfer.
PRDATA	Slave interface	The selected slave drives this bus during read cycles when PWRITE is LOW. This bus can be up to 32-bits wide.
PSLVERR	Slave interface	This signal indicates a transfer failure.

A. Write Transactions[2]

The write transaction is shown in figure 3 and it begins with the rising edge of the clock at T1 by registering PADDR, PWDATA, PWRITE, and PSEL. In the next rising edge, the PENABLE and PREADY are registered. When PENABLE is asserted at T3, it means that there is a transfer pending and if the transfer happens, PREADY implies that the slave can complete the transfer at the next rising edge of the clock.

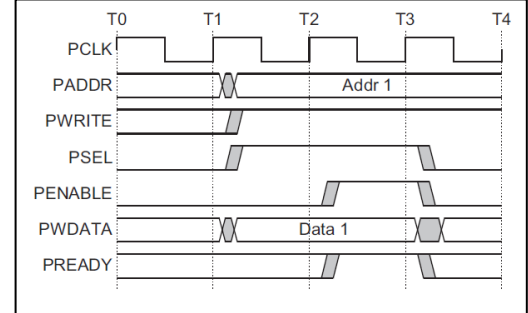


Figure 3. Write transaction with no wait state

B. Read transactions[2]

The read transaction is represented in figure 4. It begins with the rising edge of the clock at T1 similar to the write cycle. The PADDR, PWRITE, and PSEL are registered. In the next rising edge, the PENABLE, PRDATA, and PREADY are registered. When PENABLE is asserted at T3, it means that there is a transfer pending and if the transfer happens, PREADY implies that the slave can complete the transfer at the next rising edge of the clock and the data is read before the end of the transaction, controlled by the slave.

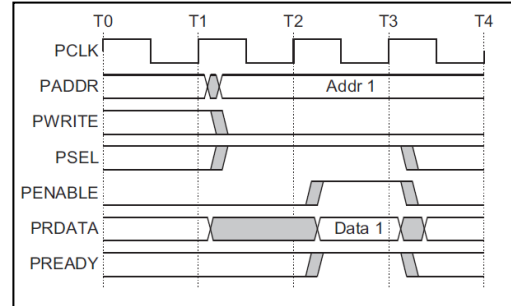


Figure 2. Read transaction with no wait states

IV. AXI2APB BRIDGE ARCHITECTURE

In the proposed design, a communication between an AXI4-Lite and APB interfaces will be realized as stated before. The architecture belonging to the proposed design is shown in figure 5.

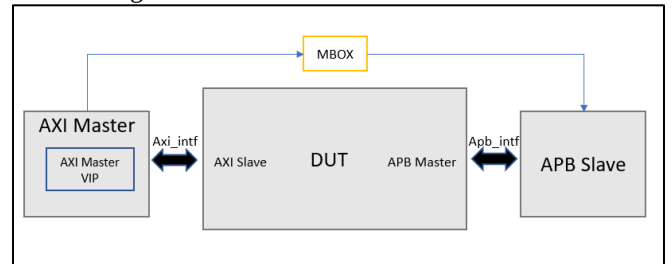


Figure 53. The proposed Axi2apb Bridge Architecture

As seen in figure, the proposed architecture has basically 3 blocks which are AXI Master, APB Slave and DUT. The

AXI Master block includes the AXI Master VIP generated using XILINX IP [5] and it is responsible to generate write and read transactions and send them to the APB Slave block through a mailbox. In the APB Slave block, the transaction data is fetched from the mailbox and controlled whether they are correct or not. The DUT is the RTL file which includes both AXI Slave and APB Master signals. In this file the connection between AXI4-Lite and APB interface is made and the signals that are needed to be driven is arranged based on the handshake process between AXI and APB interfaces. There are 2 interfaces between these blocks that provide the communication with the testbench.

According on the proposed structure, the state diagram for the operating states of the bridge is shown in figure 6. As stated before, the transition between states is arranged according to the handshake process between valid and ready signals and the read and write orders are dependent on controlling the PREADY signal.

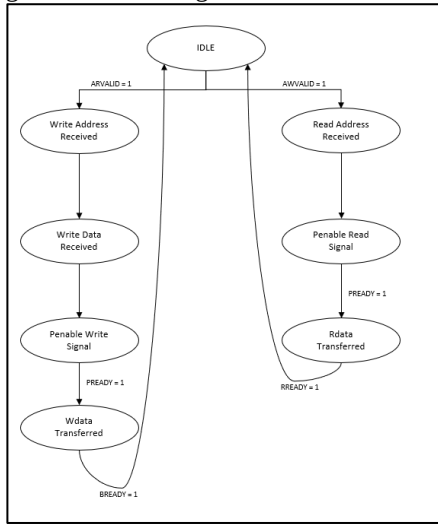


Figure 64. State Diagram of Proposed Axi2apb Bridge Protocol [3]

In the proposed Bridge architecture, when both write and read transfers are simultaneously requested on the AXI4-Lite interface, the read request has higher priority than the write request.

The state machine operates through the following states:

IDLE: This is the default state of the APB.

Write Address Received: The bus moves into the Write Address Received state when a Write transfer request is registered.

Write Data Received: The PSEL and PWRITE signals are asserted in this state. The bus moves into the Write Data Received state and waits for one clock period and directly moves to the Penable Write Signal state.

Penable Write Signal: The PENABLE signal is asserted in this state. If PREADY is asserted, the bus moves into the Wdata Transferred state.

Wdata Transferred: The BVALID signal is asserted in this state. The bus waits for BREADY to be asserted to complete the write transaction.

Read Address Received: The PSEL signal is asserted in this state. The bus moves into the Write Address Received state when a Read transfer request is registered.

Penable Read Signal: The PSEL and PENABLE signals are asserted in this state. If PREADY is asserted, the bus moves into the Rdata Transferred state.

Rdata Transferred: The RVALID signal is asserted in this state. The bus waits for RREADY to be asserted to complete the read transaction.

In the coming parts, the designed architecture will be tested and verified using AXI VIP (Verification IP) that provides example testbenches and tests for AXI3, AXI4, and AXI4-Lite interfaces. Note that the design file is created using VHDL language and simulation files are created using SystemVerilog language.

V. SYSTEMVERILOG VERIFICATION METHODOLOGY

To verify the overall system, SystemVerilog verification methodology will be used. According to this method, there are several blocks that arranges the flow of the verification as seen in figure 7. This environment checks whether the functional flow of the main design code under Design Under Test (DUT) is working properly. This environment in general verifies the design by generating stimulus and controlling the design outputs by monitoring the outputs.

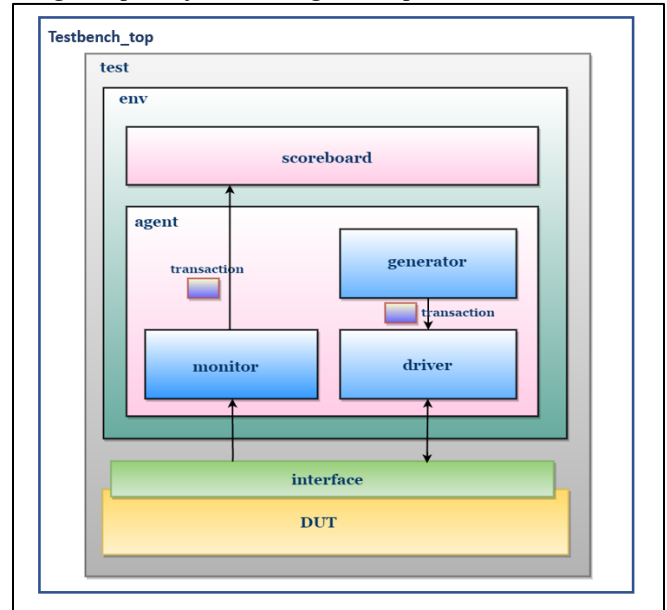


Figure 7. SystemVerilog Verification Flowchart [4]

All these blocks have special duties to verify the system and the descriptions belonging to each block is represented in table 3.

Table 3. SV Verification Testbench Components [4]

Name	Type	Description
transaction	class	Includes packets or frames that will be transferred through the tb environment which generally includes values that will be randomized and covered in the further stages.
generator	class	Generates random input stimulus that will be monitored by DUT and sends them to Driver.
driver	class	Gathers the generated stimulus with the help of mailbox and drives them.
monitor	class	Monitors the pin level activity on design input and output ports and then send them to scoreboard.
agent	class	Contains the class's (generator, driver, and monitor) specific to an interface or protocol.

scoreboard	<i>class</i>	Checks the output data items with the expected values.
environment	<i>class</i>	Contains all the components including transaction, generator, driver, monitor, and scoreboard.
test	<i>program</i>	Configures the tb and initiates the tb components
interface	<i>interface</i>	Includes signals that will be driven or monitored.
tbench_top	<i>class</i>	Topmost file that connects design with the testbench. It covers all the environment mentioned above

The proposed bridge protocol will be verified using SystemVerilog Verification Methodology with the help of AXI VIP provided by XILINX. The randomized stimulus will be generated and checked by the elements of the SV Verification methodology and the design will be validated.

VI. SIMULATION RESULTS OF THE AXI2APB BRIDGE PROTOCOL USING QUESTASIM SOFTWARE

As stated before, the AXI Verification IP which generates the stimulus for the AXI interface will be used to verify the RTL code. In addition to that the APB part of the design code will be driven and checked based on the proposed Axi2apb Bridge Architecture, presented in figure 5.

In the beginning, the address and data for the read and write transactions signals will be selected. Then in an AXI master class, these signals will be randomized and AXI4-Lite write and read transaction tasks will be generated. After that based on a write selection signal coming from the transaction class, the write or read transaction will be sent to the APB slave class using a mailbox. The received signals will be checked and the outputs of the APB slave signals will be driven inside the APB slave class. At the same time, the communication between the AXI and APB with DUT will be held on and the complete system will be monitored. The SystemVerilog Verification architecture will be used from the beginning to the end of the process.

After arranging all the blocks and importing the AXI Verification IP libraries to the project, the system is simulated using QuestaSim Software which is used to verify HDL languages. In below, you can see the simulation results for the proposed design

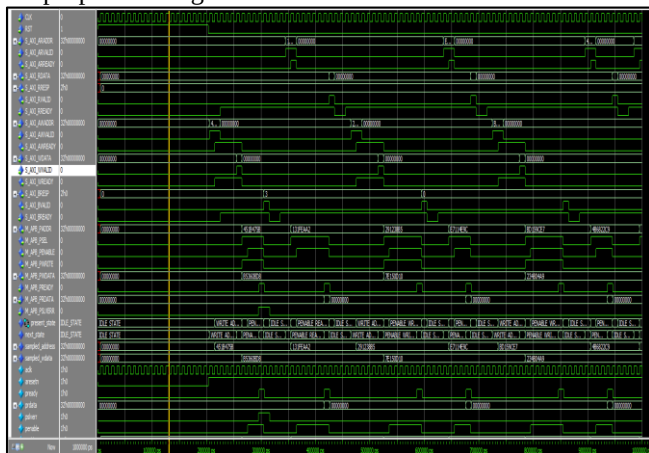


Figure 8. Simulation results of the AXI2APB Bridge protocol using QuestaSim Software

The read and write transactions for the bridge architecture is shown in figure 8. When the ARVALID,

RVALID, AWVALID, WVALID signals asserted indicating data the address or data is registered to the signals. The transactions happen according to the rules of write and read transaction of the AXI-Lite interface and the RVALID and BVALID signals are asserted based on the situations of PSEL, PEN, PWRITE, and PREADY signals.

After simulating the AXI2APB Bridge protocol, functional coverage, code coverage, and SV Assertions (SVA) are used to check all possible situations for the entire system. Functional coverage is used to measure how many of the scenarios of the design are validated/tested. SVA and coverage are also used to see the behaviour of the design while validating. Code coverage is used to measure the efficiency of the RTL code while generating stimulus to test and verify the system. In VLSI, these methods are used to check the functionality of the entire system. Therefore, it is a good measure to use coverage and assertions to get comprehensive feedback from your designs.

In figure 9, you can see the coverage summary belonging to the proposed bridge protocol that is validated using all the possible coverage methods. The FSM statements and transitions, if else clauses, signal toggles are examined and monitored to increase the functionality of the written code and necessary changes are made based on these results.

Design Unit #	Assertions	Branches	Directives	FSM States	FSM Transitions	Coverage	Statements	Toggles	Total
Search...	Search...	Search...	Search...	Search...	Search...	Search...	Search...	Search...	Search...
workapb_intf	-	-	-	-	-	-	100%	99.51%	99.75%
workapb_bridge	-	99.71%	-	100%	64.10%	-	97.36%	99.84%	91.64%
workapb_intf	100%	-	100%	-	-	-	100%	98.25%	99.56%
workapb_trans	-	-	-	-	-	-	100%	-	100%
workapb_vip_bridge	77.51%	49.75%	-	-	-	-	56.81%	5.04%	47.20%
workapb_vip_f	-	99.7%	-	-	-	-	70.8%	23.43%	51.21%
workapb_vip_master	-	-	-	-	-	-	100%	-	100%
workapb_vip_slave	100%	24.56%	-	-	-	-	50.94%	-	58.5%
workapb_vip_slv_top	-	30%	-	-	-	-	100%	5.4%	31.8%
workapb	-	-	-	-	-	-	100%	100%	100%
workapb_slv_unit	100%	100%	-	-	-	91.64%	100%	-	98.15%

Figure 9. Coverage Summary by Design Units

In the coverage summary shown above, the AXI VIP classes and interfaces are also included which have some extra signal extensions compared to the designed RTL code. Since the design doesn't cover all the AXI4-Lite interface signals such as AWPROT and ARPROT, the modules that are used to verify the RTL have a lower percentage in terms of coverage. Additionally, notice that only the codes that are written by hand should be examined while investigating the coverage results, including axi2apb_bridge, axi_intf, axi_trans, and tb. Therefore, the coverage ratio of the remaining classes has a lower percentage compared to the written modules.

VII. HARDWARE RESULTS OF THE AXI2APB BRIDGE PROTOCOL USING VIVADO

After completing the verification of the AXI2APB Bridge protocol using the simulation environment, the RTL code is tested on an FPGA board to observe further tests on the hardware. The Arty A7-35T development board is used to debug the proposed architecture and VIVADO block design is used to arrange the necessary modules and IP blocks.

In the beginning, the JTAG to AXI Master customizable IP core is used as the AXI Master which generates AXI read and write transactions and drives the AXI4-Lite interface signals internal to the FPGA [5]. Then the proposed bridge core is placed by packaging an APB Master interface and the

APB Slave module is added by packaging an APB Slave interface since these blocks are connected via interfaces. It is important to mention that the RTL code is written with synchronous active high reset and sequential order. The block design of the proposed bridge protocol is shown in figure 10. The system is checked by creating read and write transactions based on the JTAG2AXI Master IP Core [6], which found as build-in in XILINX [6]. The final results showed that the data sent is written to the address and read back from the address correctly.

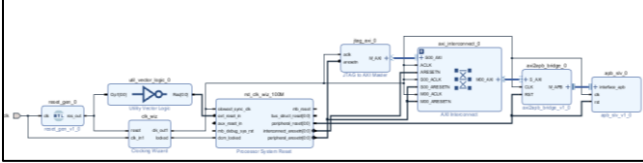


Figure 10. Block Design of the AXI2APB Bridge Communication Protocol

The list of clocks generated for the design is shown in table 4. Since the IP Core is designed based on the synchronous circuitry idea, there is only one global clock generated for the hardware.

Table 4. List of clocks

Name	Source	Rates (MHz)			Description
		Max	Min	Resolution	
sys_clk_pin	input	100	-	-	Global system clock generated for the CLK of the DUT.

The timing summary generated by VIVADO for the proposed IP Core is represented in table 5. Timing analysis uses estimated net delays that represent ideal placement and it is important to check timing closure to understand whether the design has met all the timing checks or not. The timing summary report provides an overall interpretation in terms of setup, hold, pulse-width checks and the results is observed as represented in table.

Table 5. Design Timing Summary

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.230 ns	Worst Hold Slack (WHS): 0.007 ns	Worst Pulse Width Slack (WPWS): 3.000 ns
Total Negative Slack (TNS): 0 ns	Total Hold Slack (THS): 0 ns	Total Pulse Width Negative Slack (TPWS): 0 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0

In the next segment, the AXI APB Bridge IP designed by XILINX is used to compare and analyze the effectiveness of the proposed IP Core. The post-implementation resource utilization outcomes belonging to the designed RTL and the XILINX IP are shown in table 6 and table 7, respectively. In table 6, you can see the resource usage of the proposed project. The biggest difference between the proposed IP and the XILINX IP is the register usage preference. In the proposed project the registers are made of Flip Flops instead of Latches to prevent latch inference that doesn't meet the sequential synchronous logic and causes problems such as

noise and timing failures. Also, BRAM resources are used to store data during transmission. Therefore, it is clear that the use of LUT and FF drops significantly when comparing the proposed IP with the XILINX IPs.

Table 6. Post-implementation resource utilization for the designed Bridge IP Core

Resource	Utilization	Available	Utilization %
LUT	4375	20800	21.03
LUTRAM	2843	9600	29.61
FF	2605	41600	6.26
BRAM	6.50	50	13.00
IO	1	210	0.48
BUFG	3	32	9.38
MMCM	1	5	20.00

Table 7. Post-implementation resource utilization for the XILINX IP Core

Resource	Utilization	Available	Utilization %
LUT	6899	20800	33.17
LUTRAM	2843	9600	29.61
FF	11027	41600	26.51
BRAM	6	50	12.00
IO	1	210	0.48
BUFG	3	32	9.38
MMCM	1	5	20.00

Also, the APB slave part is designed by inferring single port BRAM to increase the inspectability of place and route of the design on the FPGA, since the proposed bridge protocol has 32 wide address arrays. Additionally, setup and hold time which is a very important concept to see the amount of time for signals to pass between two Flip-Flops without having metastability are considered while designing the project.

VIII. CONCLUSION

In this study, the design and verification of the AXI2APB Bridge communication protocol is realized using VHDL for the RTL design, SystemVerilog for the verification and VIVADO Block Design for the Hardware tests. The final results showed that the proposed architecture works as expected without no transmission problem and the system can realize the write and read transaction between the AXI and APB buses with no error.

REFERENCES

- [1] ARM "AMBA® AXI™ and ACE™ Protocol Specification" datasheet, Retrieved 2011
- [2] ARM "AMBA™ 3 APB Protocol v1.0" ARM IHI 0024B Specification datasheet, Retrieved 2004
- [3] Chenghai Ma, Zhijun Liu, & Xiaoyue Ma. (2011). Design and implementation of APB bridge based on AMBA 4.0. 2011 International Conference on Consumer Electronics, Communications and Networks (CECNet), 193–196. <https://doi.org/10.1109/CECNET.2011.5768692>.
- [4] Systemverilog tutorial for Beginners. Verification Guide. (2020, April 8). Retrieved March 7, 2022, from <https://verificationguide.com/systemverilog/systemverilog-tutorial/>
- [5] "AXI Verification IP v1.1 LogiCORE IP Product Guide" Specification datasheet (pg267), Retrieved 2021
- [6] JTAG to AXI Master v1.2 datasheet (pg174), Retrieved 2021